Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete source code is attached separately in the GitHub repository and submitted along with this report.

GitHub Repository:
`https://github.com/Sadhana2006-art/Deep-Learning-Lab`

7. Experimental Results

0.1 Histograms

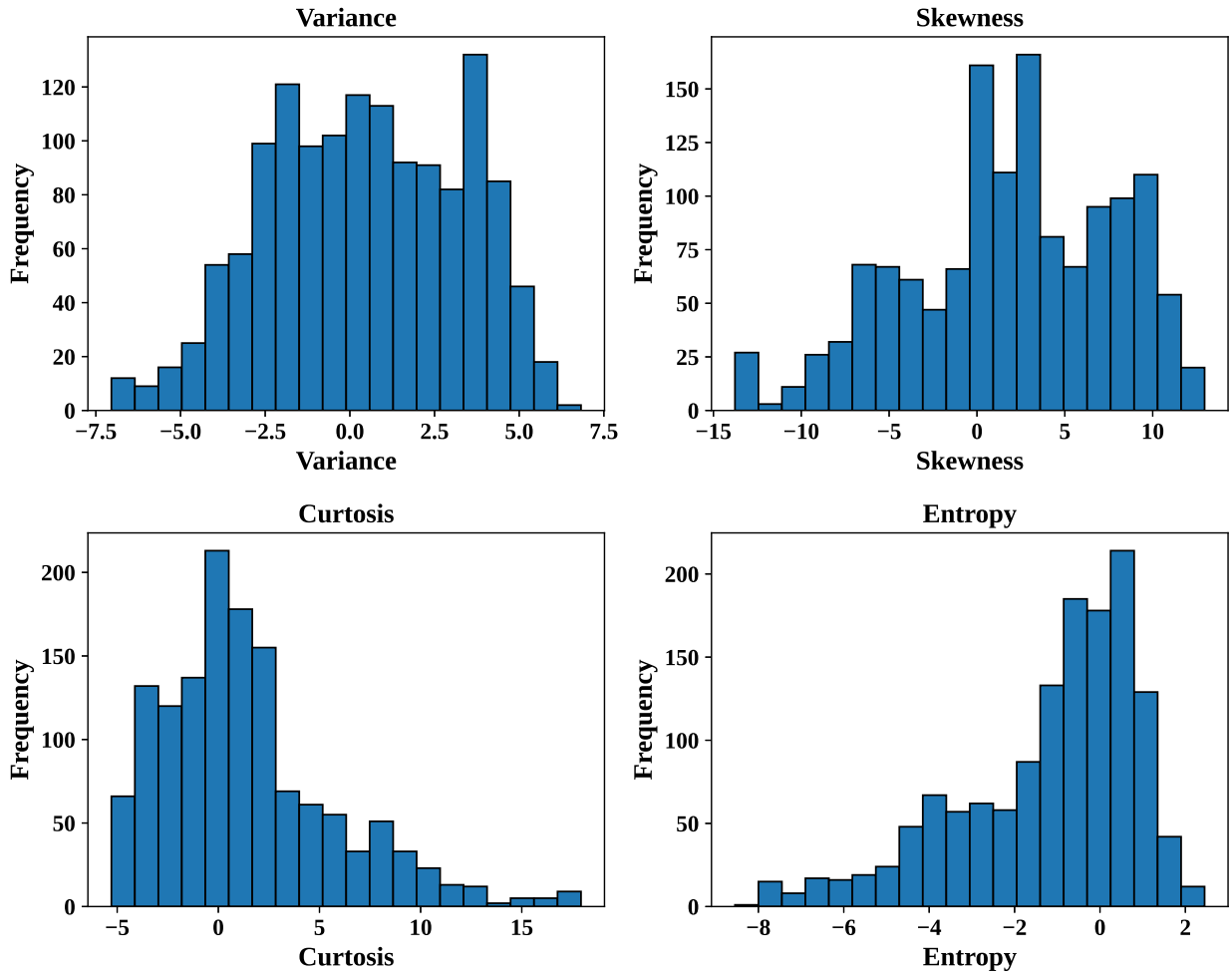

Figure 1: Histograms of Dataset Features

Interpretation:

The Variance feature is distributed on negative and positive values. Skewness has multiple peaks showing different groups of data. Curtosis is positively skewed with long right tail, Entropy is negatively skewed with longer left tail. The different distributions and ranges imply that the features need to be normalised before training the perceptron.

0.2 Correlation Heatmap

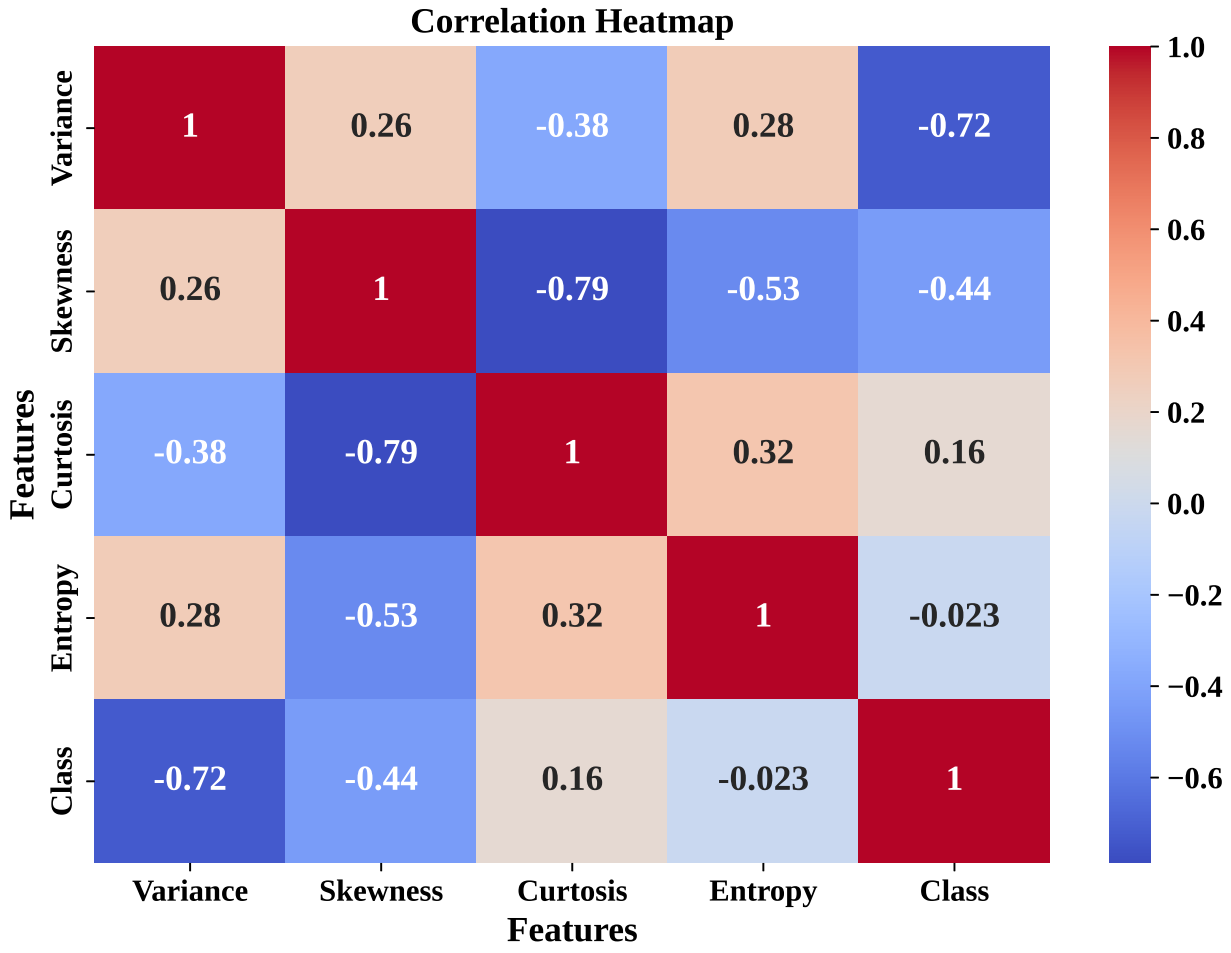

Figure 2: Correlation Heatmap

Interpretation:

The heatmap shows a strong negative correlation of Variance and Class (-0.72), suggesting that Variance is an important feature for classification. Skewness and Curtosis are also highly negatively correlated (-0.79) and Entropy is very weakly correlated with the target class.

0.3 Scatter Plot

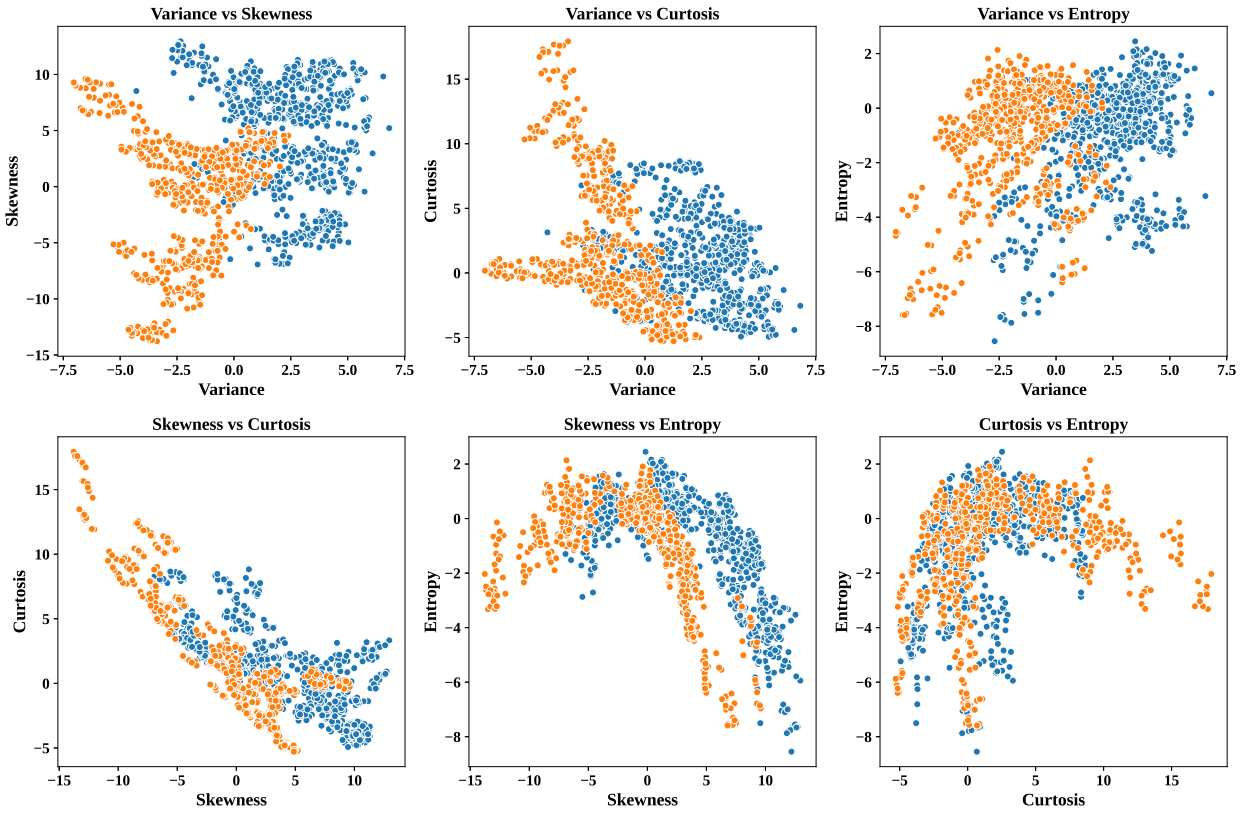

Figure 3: Scatter Plot of Feature Combinations

Interpretation:

Variance combinations of features show the clearest class separation, but Curtosis vs Entropy has relatively more overlap. The dataset seems to be quite linearly separable.

0.4 Boxplots

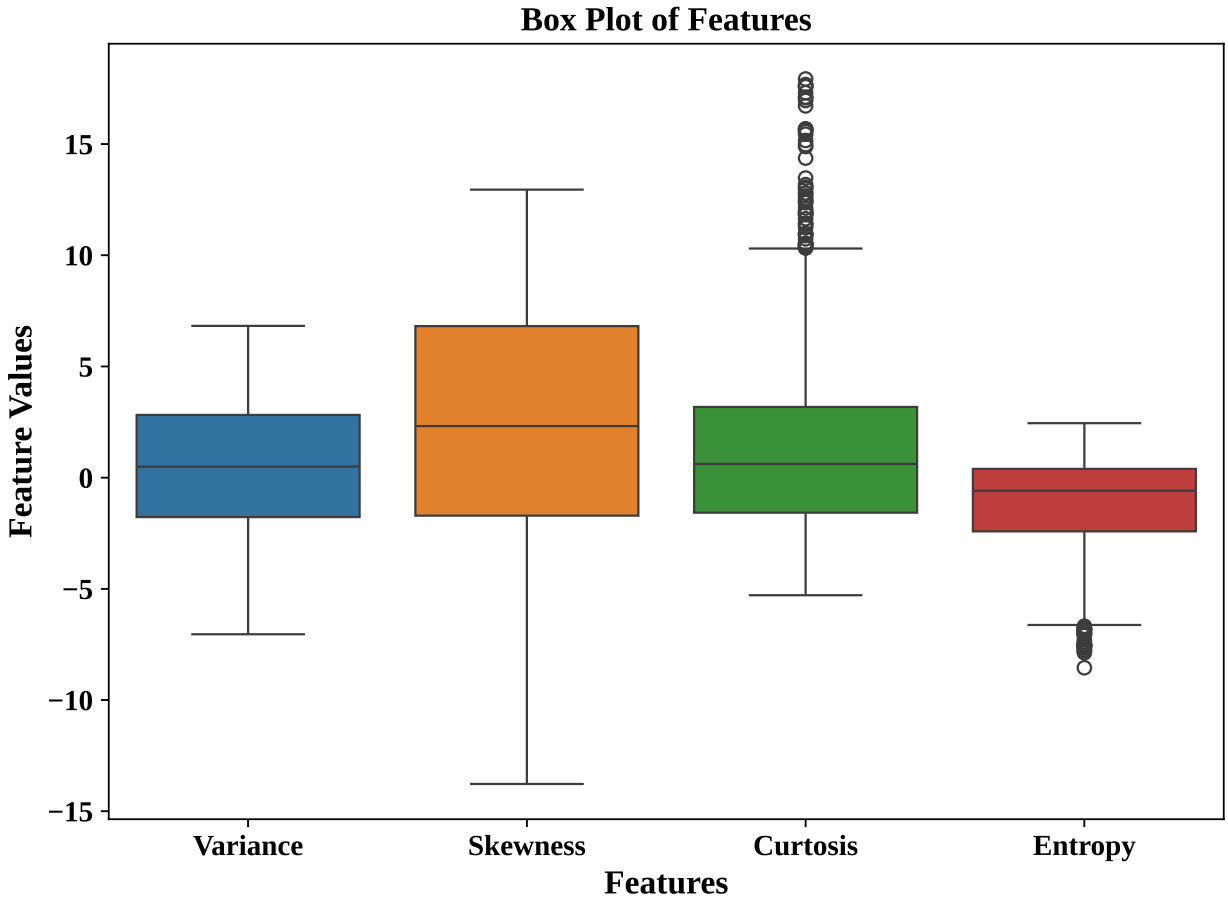

Figure 4: Box Plot of Dataset Features

Interpretation:

Box plots indicate the difference in the range and spread of features. The maximum spread is observed in the case of skewness, and there are some outliers present in the Curtosis and Entropy features.

0.5 Confusion Matrix

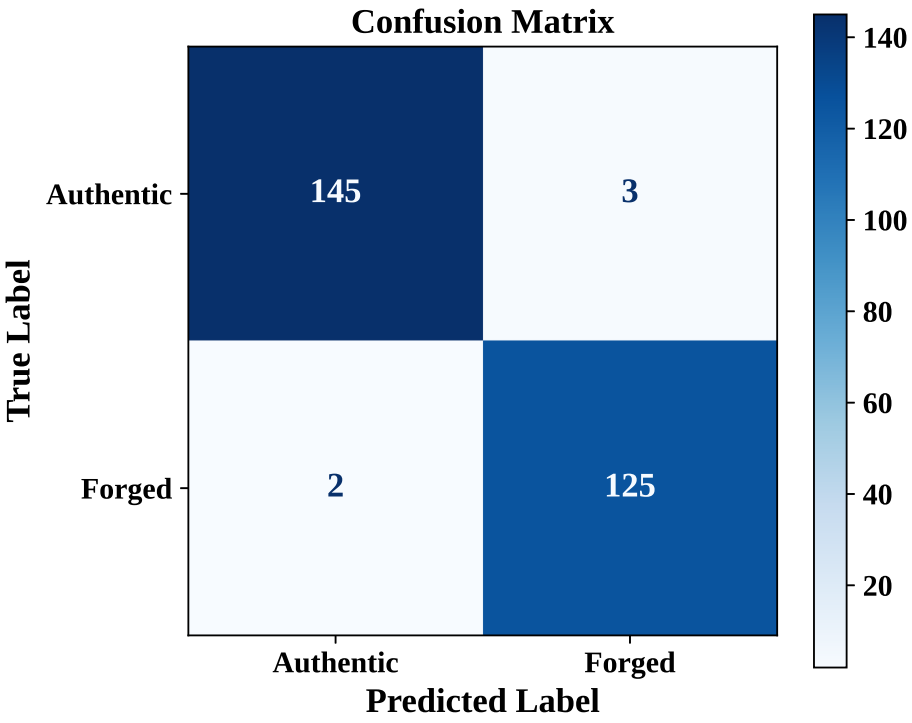

Figure 5: Confusion Matrix

Interpretation:

Confusion matrix indicate that the perceptron was able to correctly classify 145 (TN) authentic banknotes as well as 125 (TP) forged banknotes. The classifier only made 5 errors out of the total 275 banknotes (3 (FP) authentic identified as forged and 2 (FN) forged as authentic).

0.6 Training Error vs Epoch

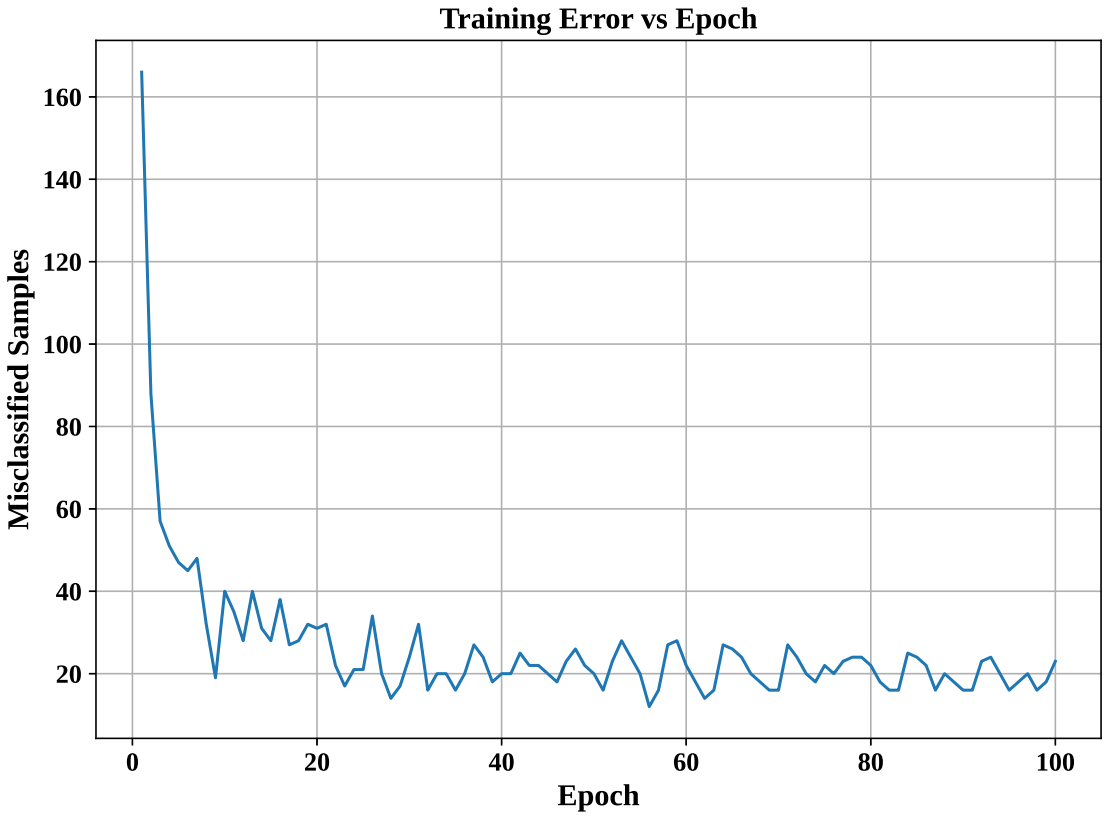

Figure 6: Training Error versus Epoch

Interpretation:

The value of the training error reduces very fast at the beginning because the perceptron manages to learn the decision boundary. Once the number of epochs reaches about 20, the error starts to oscillate between 15 and 30 samples and does not reduce further. Thus, we can say that the perceptron has not fully achieved convergence.

0.7 Weight Evolution

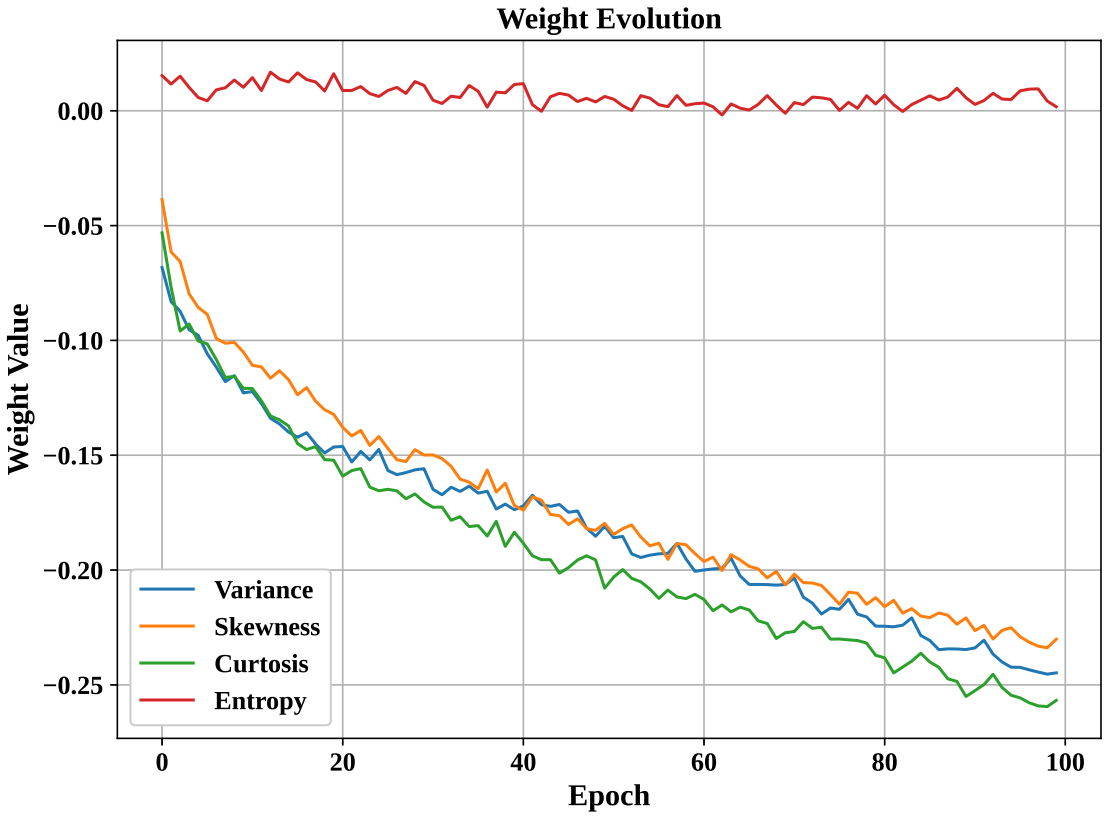

Figure 7: Evolution of Perceptron Weights

Interpretation:

There is considerable variation in the weights in the beginning stages of learning since the perceptron learns from the data provided to it during training. However, as training proceeds, the weights become stable with minor variations, suggesting that the model has learned a stable set of parameters.

0.8 Bias Evolution

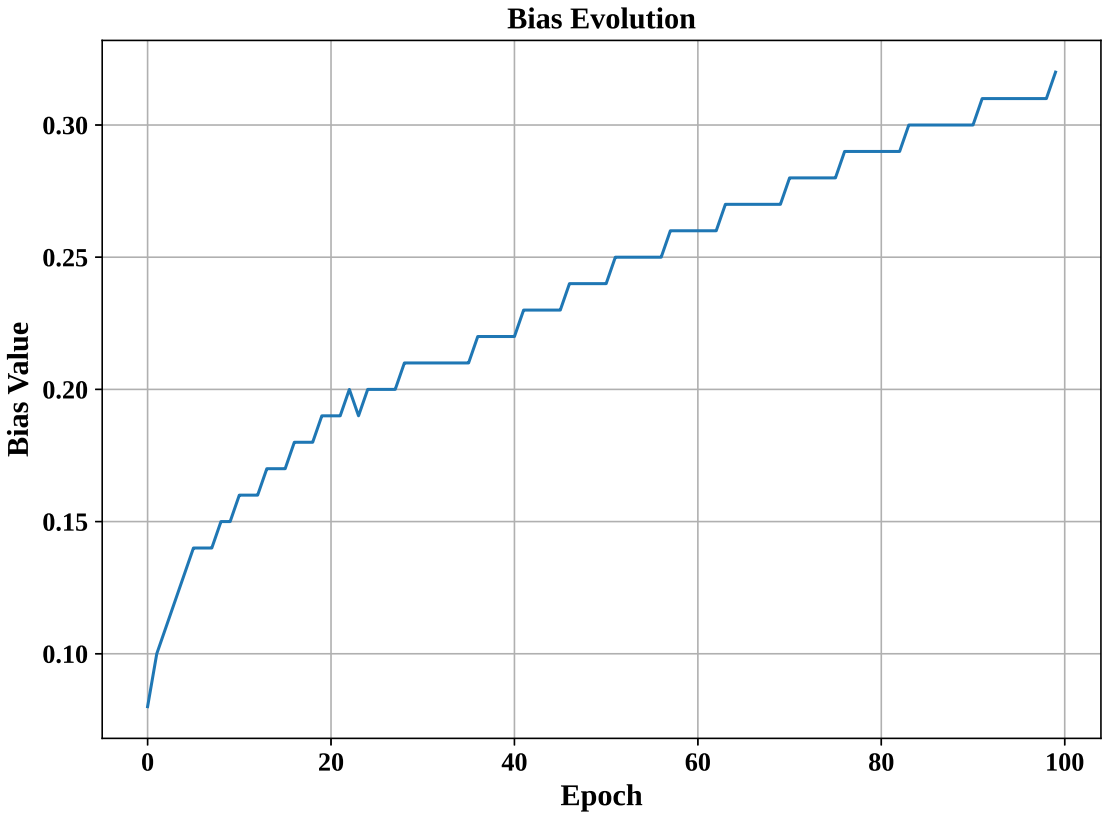

Figure 8: Evolution of Bias

Interpretation:

The increase is very fast in the early epochs, and then it becomes slower as time goes by. The stepping pattern shows that the bias only changes when there are misclassified samples, and it finally converges to 0.32, showing that the model has already learned a proper threshold for classification.

0.9 Learning Rate Comparison

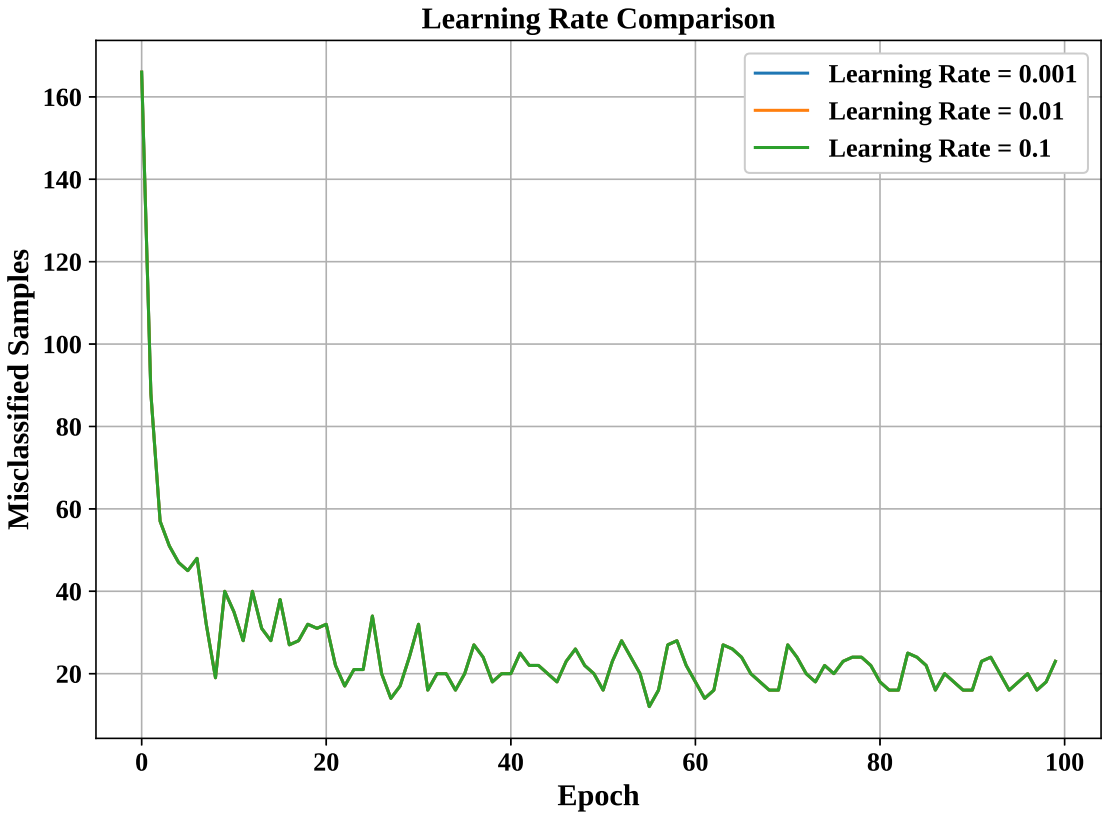

Figure 9: Comparison of Different Learning Rates

Interpretation:

The same curve is observed for all learning rates (0.001, 0.01, and 0.1) since perceptron weights are updated only when a sample is misclassified. As the data is almost linearly separable and has been scaled using MinMaxScaler, the same samples are misclassified every epoch. The only thing affected by learning rates is the magnitude of the change in the weights.

8. Performance Tables

Training Summary

Dataset Size	1372 Samples
Train/Test Split	1097 Samples (80%) and 275 Samples (20%)
Learning Rate	0.01
Epochs	100
Final Weights	-0.2447762, -0.23011114, -0.25674323, 0.00175867
Final Bias	0.3200
Accuracy	0.9818
Precision	0.9766
Recall	0.9843
F1-score	0.9804

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	166	-0.06827408	-0.03866853	0.08
2	88	-0.0831161	-0.06146315	0.0999
3	57	-0.08731574	-0.0656554	0.1099
4	51	-0.09538568	-0.07974311	0.1199
5	47	-0.09778344	-0.0856403	0.1299
⋮	⋮	⋮	⋮	⋮
100	23	-0.2447762	-0.23011114	0.3200

9. Discussion

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

   Step Activation Function gives the output as either 0 or 1, while the Sigmoid Activation Function gives an output that varies continuously from 0 to 1. The Step function is never used in deep learning since it is not differentiable and hence cannot be used in backpropagation. This is the reason why Sigmoid and other activation functions are used.

2. **Compare the implementation with Scikit-learn's Perceptron.**

   The perceptron that has been developed is an implementation of the basic perceptron learning rule, where weights and biases are adjusted manually. The Scikit-learn Perceptron, on the other hand, is highly optimized and incorporates functions like shuffling data, early stopping, and regularization.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

In this case, the experiments were done using the learning rates of 0.001, 0.01 and 0.1. All three learning rates converged similarly and performed almost equally well. The learning rate impacted mainly the size of weight changes rather than affecting the accuracy of classification.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

   Single Layer Perceptron can classify only linearly separable data. As the XOR problem is not linearly separable, a single decision boundary cannot distinguish its classes. Hence, Multilayer Perceptron is needed for solving the XOR problem.

5. **Study the effect of feature normalization on model convergence.**

   Feature scaling will ensure that all input features have a standard scale value, thereby avoiding the domination of any feature with higher value. In the current experiment, MinMax scaling will increase training stability and allow for faster convergence of the perceptron.

10. Additional tasks

0.10 Perceptron learning algorithm for OR Gate

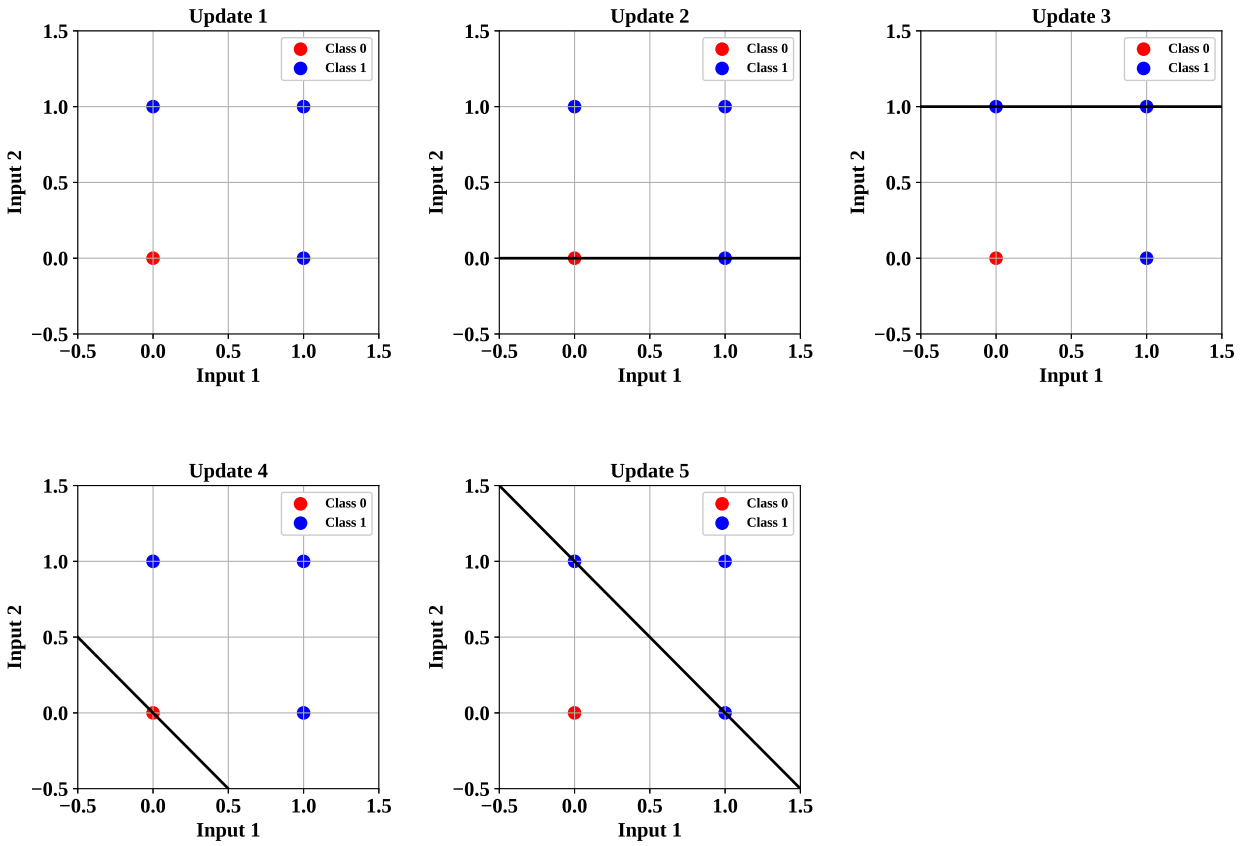

Figure 10: OR Gate Decision Boundaries

Interpretation:

The decision boundary is changing after each weight update, and it gradually separates the OR gate inputs into correct classes and shows the perceptron learning process.

0.11 Perceptron learning algorithm for AND Gate

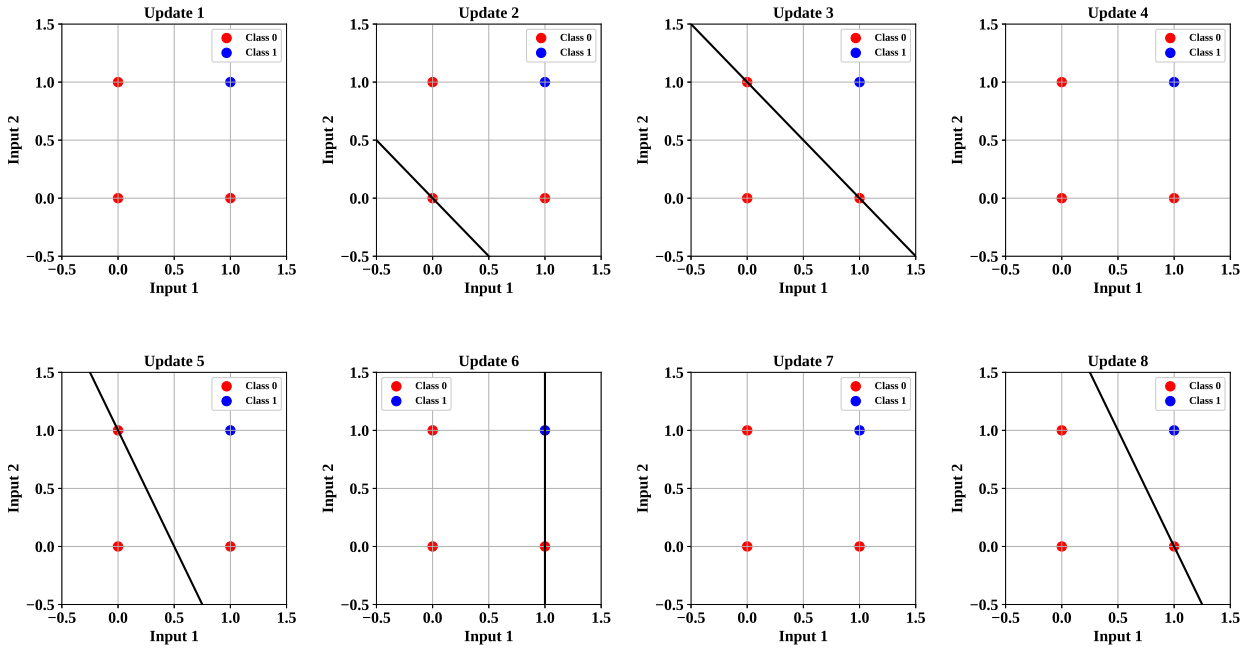

Figure 11: AND Gate Decision Boundaries

Interpretation:

The decision boundary is changing after each weight update and gradually moving to separate the single positive class (1,1) from the rest negative samples, showing the perceptron learning process.

0.12 Perceptron learning algorithm for NOT Gate

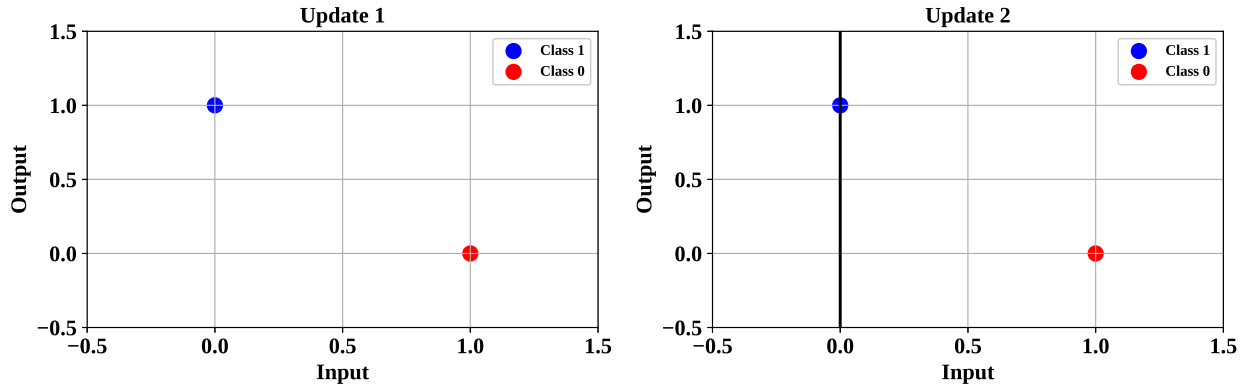

Figure 12: NOT Gate Decision Boundaries

Interpretation:

The perceptron learns the NOT gate in just 2 weight updates. This means that the data is

linearly separable. After the second update the decision boundary quickly stabilises and both input samples are correctly classified with successful convergence.

11. Conclusion

In the experiment, the Single Layer Perceptron was successfully developed for the purpose of classifying the Banknote Authentication dataset into two classes. For this, the dataset was normalized through MinMax technique while the perceptron learning algorithm and step activation function were used to train the classifier. As a result, the classifier performed extremely well on linearly separable data.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.